# RISC-V Memory Tagging Extension

Version v0.2, 2025-09-08: Draft

# Table of Contents


List of figures . . . 1
List of tables . . . 2
List of listings . . . 3
Copyright and license information . . . 4
Contributors . . . 5
1. Introduction . . . 6
2. Memory tagging extension (Zimt) . . . 8
2.1. Taxonomy for loads and stores . . . 8
2.1.1. Regular load and regular store operations . . . 8
2.1.2. Checked load and store . . . 8
2.1.3. Unchecked load and store . . . 8
2.1.4. Load tag and store tag operations . . . 9
2.2. Tag widths . . . 9
2.3. Tag placement in pointer . . . 9
2.3.1. Generate a tag - gentag rd . . . 9
2.3.2. Arithmetics on pointer tag - addtag rd, rs1, #tag_imm4 . . . 10
2.4. set tag(s) for memory chunk(s) . . . 10
2.4.1. Store tag(s) for memory chunk(s): settag rs1, #chunk_count . . . 10
Memory ordering requirement . . . 11
Exceptions . . . 11
2.5. tag checks and privilege modes . . . 11
2.5.1. M-mode . . . 11
2.5.2. Less than M-mode . . . 11
2.5.3. tag checks . . . 12
2.5.4. Explicit tag check . . . 12
2.6. Asynchronous reporting for tag mismatches . . . 12
2.7. Tag checks on page basis . . . 12
2.7.1. Tag checks on supervisor accesses to user pages . . . 13
2.8. Memory tagging and hypervisor memory accesses . . . 13
2.9. CSR bits for memory tagging . . . 13
2.9.1. Memory tagging enable and pointer_tag_width . . . 14
2.9.2. Machine Security Configuration Register(`mseccfg`) . . . 14
2.9.3. Machine Environment Configuration Register (`menvcfg`) . . . 14
2.9.4. Supervisor Environment Configuration Register (`senvcfg`) . . . 15
2.9.5. Hypervisor Environment Configuration Register (`henvcfg`) . . . 15
2.9.6. Hypervisor Status Register (`hstatus`) . . . 16
2.10. Appendix . . . 17
2.10.1. Example: stack tagging codegen . . . 17
3. Virtually indexed tag storage (Svatag and Smvatag) . . . 18
3.1. Tag look up in virtually indexed tag table . . . 18

3.2. Tag table base address CSRs . . . 18
3.3. Protection of tag storage . . . 19
4. tag check elide extension . . . 20
4.1. CSR bits for elide tag check . . . 20
4.1.1. Machine Status Register (`mstatus`) . . . 20
4.1.2. Supervisor Status Register (`sstatus`) . . . 21
4.1.3. Virtual Supervisor Status Register (`vsstatus`) . . . 21
Bibliography . . . 22

# List of figures

Figure 1. Machine security configuration register(`mseccfg`)
Figure 2. Machine environment configuration register (`menvcfg`)
Figure 3. Supervisor environment configuration register (`senvcfg`)
Figure 4. Hypervisor environment configuration register (`henvcfg`)
Figure 5. Hypervisor status register (`hstatus`)
Figure 6. Machine-mode status register (`mstatus`) for RV64
Figure 7. Supervisor-mode status register (`sstatus`) when `SXLEN=64`
Figure 8. Virtual supervisor status register (`vsstatus`) when `VSXLEN=64`

# List of tables

Table 1. `pointer_tag` and `mc_tag` valid configurations
Table 2. `MT_MODE` encoding and its meaning
Table 3. `mc_tag_va` for each `mc_tag_width`
Table 4. Privileged tag base address CSRs
Table 5. Lowest and highest virtual addresses

# List of listings

*This document is in the Development state*

*Expect potential changes. This draft specification is likely to evolve before it is accepted as a standard. Implementations based on this draft may not conform to the future standard.*

# Copyright and license information

This specification is licensed under the Creative Commons Attribution 4.0 International License (CC-BY 4.0). The full license text is available at creativecommons.org/licenses/by/4.0/.

Copyright 2024 by RISC-V International.

# Contributors

This RISC-V specification has been contributed to directly or indirectly by (in alphabetical order):

Adam Zabrocki, Alvin Che-Chia Chang, Andrew Waterman, Brendan Sweeney, Christoph Müllner, David Weaver, Deepak Gupta, Earl Killian, Florian Mayer, Luís Fiolhais, Martin Maas, Nick Kossifidis, Radim Krčmář, Ravi Sahita, Robbin Ehn, Samuel Holland, Siqi Zhao, Thurston Dang, Vedvyas Shanbhogue

# Chapter 1. Introduction

Significant production software is written in languages which provide powerful memory management primitives (like C/C++). Software is written in collaborative fashion and on tight deadlines and bugs do get introduced. A subset of these bugs introduce issues commonly termed as "memory safety" issues. The root causes of memory safety issues are numerous but eventually such issues surface as invalid memory accesses with regards to the language specification.

Programs use pointers to access memory. We refer to the parts of the programs in which accessing an object is valid as its "lifetime". An object's lifetime depends on the region of memory it is stored on:

- **Heap**: A memory allocator uses operating system facilities to create large memory mappings and expose APIs so that software can manage (allocate and free) memory at their convenience. As an example `malloc` allocates a specified number of bytes and returns a pointer to the allocated memory; `free` frees the memory so that allocator can recycle that memory. Depending on the operating system, there might be many similar APIs (e.g. `calloc`, `memalign`, etc.), but they are always paired with a corresponding `free` call. The lifetime of an object on the heap is between `malloc` (or similar) and `free`.
- **Stack**: The operating system allocates a region to be used as the stack of the program. When a function is called, a stack frame is created at the top of the stack. This is used to hold local variables, which are accessed relative to the stack pointer, which designates the top of the stack. Upon return from the function, the frame is popped from the stack by adjusting the stack pointer. Within a stack frame, the compiler can reuse the same memory for local variables that have non-overlapping lifetimes (known as stack coloring, implemented e.g. in LLVM StackColoring [1]). The lifetime of an object on the stack is governed by the programming language's scoping rules (e.g. C++ scoping rules [2]).
- **Globals**: Storage for global variables is allocated by the operating system on process startup (or when the dynamic object is loaded). Generally, the storage for globals is destroyed on process shutdown (depending on the OS, `dlclose` might cause it to be destoyed earlier). The lifetime of a global ends when the process is terminated, or the dynamic object is unloaded.

In addition, an object has a size that represents the amount of memory allocated for it, and except for reassignment, a pointer must always point to memory of the object it was previously assigned to point to.

Two prominent patterns of invalid memory accesses are:

- **Temporal**: A pointer is dereferenced past the lifetime of the object it refers to. With respect to heap memory, such issues are commonly called "use after free" (UAF). In case of stack, such issues are termed as "stack use after return", "stack use after scope".
- **Spatial**: A pointer outside of the bounds of the object it refers to be created, leading to invalid memory access when dereferenced. This is commonly referred to as a "buffer overflow" (sometimes also "buffer underflow"), or "out of bounds".

To catch these issues, we can check memory accesses for validity at runtime. We do this by employing a lock and key scheme. Memory is assigned a memory tag (lock). Each pointer is assigned a pointer tag (key) in its unused top bits. On access, the key needs to match the lock, or the program is terminated.

When the lifetime of an object starts (e.g. `malloc` is called), we assign the same tag as a memory tag (lock) for the `size` bytes representing the object, and pointer tag (key). Subsequent in-bounds accesses using that pointer are valid. When the lifetime of the object ends (e.g. `free` is called), we change the

memory tag (lock). Subsequent accesses through the pointer (which remains unchanged) will be invalid and terminate the program.

Existing software mechanisms using compiler instrumentation and runtime support exist:

- HWAddressSanitizer (HWAsan) [3] which uses pointer-masking to implement the idea explained above.
- AddressSanitizer (ASan) [4] which is fully implemented in software, but in a different way to the idea above.

However software approach has significant cost from performance perspective and are mostly limited to unit test, developer test or special test harness (like fuzzing) environments. Hardware assistance in management and checking of `tags` on memory accesses helps in more scalable testing of software in variety of deployment scenarios including production.

Leveraging this software usage model, the `Zimt` extension defines a mechanism for tagged pointer construction, tag management instructions to store tags and tag checks on memory accesses (pointer dereferences). `Svatag` and `Smvatag` extensions defines tag storage mechanism in virtual memory along with protection mechanisms for tag storage.

Software (particularly compiler) has visiblity into lifecycle of pointers and can statically determine accesses to be safe in some reduced scope. To aid compiler from eliding such tag checks, `Zitagelide` is defined.

# Chapter 2. Memory tagging extension (Zimt)

The Zimt extension divides address space of the program in equal memory chunks of 16 bytes (hereafter `mc`). Each `mc` in address space can be associated with a tag (hereafter `mc_tag`) by software and `mc_tag` for `mc` is stored in memory defined by extension Svatag (see Chapter 3).

Software (runtime memory allocator or compiler) that constructs pointers to access memory (and thus memory chunks) is expected to place a tag (hereafter `pointer_tag`) in high unused bits of the pointer as defined by the pointer masking extensions [5]. Thus `Zimt` depends on the pointer masking extension and is applicable to only for XLEN=64. Pointer masking has to be enabled as a requirement for memory tagging to be enabled in the execution environment.

To maintain binary compatibility with implementations where memory tagging extension may not be implemented or may not be enabled for a workload, instruction encodings for memory tagging are taken from zimop [6] encoding and thus make memory tagging extension dependent on zimop.

A memory tag (`mc_tag`) can be up to 8 bits wide. Thus, depending on memory chunk counts, a `load tag` or `store tag` (see taxonomy for loads and stores) may trigger a misaligned memory access exception (e.g by performing a 2-byte tag access on an odd address boundary). An implementation can avoid penalty for such misaligned tag accesses by implementing the Zicclsm extension.

*The memory tagging extension is based on the Pointer Masking extension and follows the same rules for memory access. For further information, see section 24.2.6 of the privilege specification: Memory Accesses Subject to Pointer Masking.*

## 2.1. Taxonomy for loads and stores

`Zimt` introduces new type of loads and stores to the hart and enforces tag checks on certain loads and stores. Following terminologies are defined for better clarity and will be used in the rest of the specification.

### 2.1.1. Regular load and regular store operations

Load and store operating on a page with permissions `--R`, `X-R`, `-WR` and `XWR` are termed as a `regular load` and `regular store`. Regular load and regular store memory operations are generated from an instruction (except `settag` and `checktag`) defined by ISA to perform memory accesses.

### 2.1.2. Checked load and store

Regular load and store which is subject to tag check is termed as checked load and store (or checked memory access)

### 2.1.3. Unchecked load and store

Regular load and store which is not subject to tag check is termed as unchecked load and store (unchecked memory access).

### 2.1.4. Load tag and store tag operations

`load tag` operation is a load of tag from tag storage as defined by Svatag or Smvatag extensions. `store tag` operation is a store of tag to tag storage as defined by Svatag or Smvatag extensions. `load tag` operation requires permission for tag storage memory to be --R while `store tag` operation require permission to be -WR.

Checked loads and stores shall generate load tag operation in order to fetch the tag for the memory chunk. `settag` instruction may generate a load tag followed by a store tag memory operation.

## 2.2. Tag widths

Width of the `pointer_tag` (hereafter `pointer_tag_width`) can be 4 bit or 7 bit wide depending on implementation. Discovery of `pointer_tag_width` is described in Section 2.9 section. Width of the `mc_tag` (hereafter `mc_tag_width`) is dependent on the `pointer_tag_width`. Following table describes valid configurations of `pointer_tag_width` and `mc_tag_width`.

*Table 1.* `pointer_tag` *and* `mc_tag` *valid configurations*

| pointer_tag_width | mc_tag_width |
|---|---|
| 4 | 4 |
| 7 | 8 |

## 2.3. Tag placement in pointer

Zimt defines following instructions to annotate a pointer with `pointer_tag` and perform arithmetics on `pointer_tag` in a tagged pointer. Placement and arithmetics on `pointer_tag` are performed in-place in masked bits (as defined by pointer masking extension) starting at b63. Instructions introduced to place and perform arithmetics on `pointer_tag` are encoded using `MOP.RR.1` from zimop extension.

*Existing RISC-V instructions can also be used to place (and do arithmetics)* `pointer_tag` *in masked bits of a pointer. But doing so increases code size, more so if tagging for stack memory is enabled. And new instructions help contain the code size growth.*

Following are the instructions to place `pointer_tag` in the source register

### 2.3.1. Generate a tag - gentag rd

If memory tagging is enabled in the current execution environment (see Section 2.9.1), hart clears `rd`, generates a `pointer_tag` value and places the result in `rd[XLEN:XLEN-pointer_tag_width+1]`.

If memory tagging is disabled in the current execution environment (see Section 2.9.1), then `gentag` instruction falls back to zimop behavior and zeroes destination register.

| 31 – 25 | 24 | 23 – 20 | 19 – 15 | 14 – 12 | 11 – 7 | 6 – 0 |
|---|---|---|---|---|---|---|
| 1000011 | 0 | tag_imm4 | rs1 | funct3 | rd | opcode |
| gentag | 0 | 0000 | 00000 | 100 | pointer_tag | SYSTEM |

*See Appendix for example how this can be used by codegen.*

### 2.3.2. Arithmetics on pointer tag - addtag rd, rs1, #tag_imm4

If memory tagging is enabled in the current execution environment (see Section 2.9.1), `addtag rd, rs1` instruction performs addition of `pointer_tag` specified in `rs1[XLEN:XLEN-pointer_tag_width+1]` with the unsigned value `tag_imm4` shifted left by `XLEN - pointer_tag_width` bits and places incremented `pointer_tag` value in `rd[XLEN:XLEN-pointer_tag_width+1]`.

If memory tagging is disabled in the current execution environment (see Section 2.9.1), then `addtag` instruction falls back to zimop behavior and zeroes destination register.

| 31 – 25 | 24 | 23 – 20 | 19 – 15 | 14 – 12 | 11 – 7 | 6 – 0 |
|---|---|---|---|---|---|---|
| 1000011 | 0 | tag_imm4 | rs1 | funct3 | rd | opcode |
| addtag | 0 | non-zero | tagged_pointer | 100 | pointer_tag | SYSTEM |

*addtag instruction can be used by the compiler for generating distinct pointer tags derived from a base tag (base tag obtained via gentag). Compiler can use this mechanism to assign different tags (with same base tag) for consecutive objects on stack and help catch adjacent overflow bugs. This also helps with language runtime during events like exception unwind to calculate tags for objects on stack in a deterministic manner.*

*See Appendix for example how this can be used by codegen.*

## 2.4. set tag(s) for memory chunk(s)

Zimt defines an instruction to store tag (i.e. `pointer_tag`) value(s) for consecutive 1 to 16 memory chunk(s). Base address of the first memory chunk is calculated by doing `rs1 & (~0xF)`. Count of memory chunk is encoded as 4 bit immediate (#chunk_count) in the instruction. This instruction is encoded using `MOP.RR.0` from zimop extension. Immediate encodings in #chunk_count are zero based and thus #chunk_count = 0 means first chunk and #chunk_count = 15 means 16th chunk.

*REMOVEME: after development phase of spec is done.* `gettag` *operation is omitted from extension definition because its not needed. Situations where* `gettag` *is required deemed to be not performance critical paths. Furthermore, if software desires to read tag in these non- performance critical paths, it can do so by creating an alternate read-only mapping of tag storage with the help of supervisor software.*

*Since extension is still in development phase, if* `gettag` *operation is required by software, it can be added.*

### 2.4.1. Store tag(s) for memory chunk(s): settag rs1, #chunk_count

If memory tagging is enabled in the current execution environment (see Section 2.9.1), `settag` instruction creates a `mc_tag` = `rs1[XLEN:XLEN-pointer_tag_width+1]` and generate a `store tag` memory operation with store data = `mc_tag(s)` for consecutive memory chunks encoded by `chunk_count` starting with the first memory chunk calculated from virtual address specified in `rs1`.

If memory tagging is disabled in the current execution environment (see Section 2.9.1), then `settag` instruction falls back to zimop behavior and zeroes x0, which is a no-op.

| 31 – 25 | 24 | 23 – 20 | 19 – 15 | 14 – 12 | 11 – 7 | 6 – 0 |
|---|---|---|---|---|---|---|
| 1000001 | 0 | imm4 | rs1 | funct3 | rd | opcode |
| settag | 0 | chunk_count | tagged_pointer | 100 | 00000 | SYSTEM |

*Note on tag stores*

> *When* `pointer_tag_width = 4 bit`, `mc_tag` *(stored) width is 4 bit and thus maximum width of tag store operation can be 64 bit wide (each memory chunk needs 4 bit tag and maximum possible chunks are 16. 4x16 = 64 bit). When* `pointer_tag_width = 7 bit`, `mc_tag` *(stored) width is 8 bit and thus maximum width of tag store operation can be 128 bit wide (each memory chunk needs 8 bit and maximum possible chunks are 16. 8x16 = 128 bit).*

Depending on width of `mc_tag`, `settag` may end up being a read, modify and then write operation on the memory region defined by Svatag and Smvatag extensions. There are no atomicity requirements on the implementation for `settag` instruction. If atomicity is desired for `store tag` operation then it is software's responsibility using existing mechanisms.

`settag` can generate store operations larger than maximum store width supported by implementation and implementation may choose to split it into multiple stores which follows hart's memory consistency model. There are no ordering requirements or dependencies among splitted stores.

#### Memory ordering requirement

A regular memory access (regular load or regular store) to some virtual address `va` can not bypass the older store tag initiated by `settag rs1=va`.

This specification defines tag as the entity associated to virtual addresses. In case of aliasing (multiple virtual addresses map to same physical address), it is software's responsibility to ensure that the tags are set according to software's need for respective virtual address prior to memory accesses via aliased virtual address.

#### Exceptions

`settag` can raise store page fault or access fault depending on how tag storage is oragnized. If implementation doesn't support misaligned accesses, `settag` instruction can raise misaligned exception if calculated address for locating tag is unaligned. Tag storage memory must be idempotent memory else `settag` raise store/AMO access-fault exception.

## 2.5. tag checks and privilege modes

If memory tagging is enabled for the execution environment, then memory tagging checks are performed by extracting `pointer_tag` from the input pointer and comparing `pointer_tag` against `mc_tag` associated with memory chunk for the respective address. If the effective address range of a load/store spans multiple memory chunks, the applicability of memory tag checks is determined separately for each memory chunk.

### 2.5.1. M-mode

If memory tagging is enabled in M-mode (see Section 2.9.1), all regular loads and regular stores are subject to memory tagging checks.

### 2.5.2. Less than M-mode

If memory tagging is enabled in the current execution environment (see Section 2.9.1) and `satp.MODE == Bare`, then all regular loads and regular stores are subject to tag checks.

If memory tagging is enabled in the current execution environment and `satp.MODE != Bare`, then a

regular load and regular store is subject to tag checks only if the page is marked as a tagged page (see Section 2.7) in the first stage page table.

### 2.5.3. tag checks

Once a regular load/store is determined (after paging bit checks and *envcfg control bit checks) to be subject to memory tagging checks, following further checks are performed

- All stack pointer (sp/x2) relative accesses are not checked for tags (see notes).
- Hart evaluates expression `mc_tag == pointer_tag` and if false then hart raises a software check exception with tval = 4.

While performing tag check on a regular load/store, fetching (load tag operation) `mc_tag` from the tag memory region holding tags may also result in a load page fault or load access fault and hart reports the virtual address of the tag in `xtval`.

*As much as possible, compiler uses stack pointer (x2) to access stack objects local to a function. These accesses are deemed to be safe and thus are not subject to tag checks.*

### 2.5.4. Explicit tag check

Zimt defines an explicit tag check instruction taken from `MOP.RR.1` of zimop extension, `checktag rs1, #chunk_count`. If memory tagging is disabled in the execution environment then `checktag` instruction reverts to zimop behavior. If memory tagging is enabled in the execution environment then `checktag` instruction extracts `pointer_tag` from `rs1` and performs load tag operation for memory chunks (count = `#chunk_count`) starting at virtual address `rs1`. If any of the loaded `mc_tag` value(s) mismatches with the `pointer_tag` specified in `rs1`, then software check exception is raised with tval = 4. If memory tagging is enabled in the execution environment then `checktag` instruction always makes the check irrespective of `PTE.MTAG` bit setting for the code page from where `checktag` instuction is fetched or for data page virtual address specified in `rs1`.

| 31 – 25 | 24 | 23 – 20 | 19 – 15 | 14 – 12 | 11 – 7 | 6 – 0 |
|---|---|---|---|---|---|---|
| 1000011 | 0 | tag_imm4 | rs1 | funct3 | rd | opcode |
| gentag | 0 | chunk_count | tagged_pointer | 100 | 00000 | SYSTEM |

## 2.6. Asynchronous reporting for tag mismatches

To improve performance, software check exceptions due to tag mismatches on regular stores can be reported asynchronously. This means that reported `epc` might not be the reason for tag mismatch and software must do additional analysis to infer which store resulted in software check exception. This behavior can be optionally turned on through `xenvcfg` CSR for next less privilege mode (see Section 2.9).

Note that tag check violations on regular loads must always be reported synchronously.

## 2.7. Tag checks on page basis

`Zimt` introduces `memory tag` (`MTAG`) bit in first stage page table which if set in page table entry and memory tagging is enabled from *envcfg CSR, following rules apply:

1. tag checks are enforced on regular load/store to such page (tagged data page). See Section 2.5 for

further checks. Underlying tagged page must be an idempotent memory else tag look up for referenced virtual memory will result in load access-fault exception.

2. fetched instructions from a code page with `MTAG=1` (tag-exempt code page) generate unchecked load and store. This doesn't have any impact on behavior of `settag`, `gentag` and `addtag` instructions.
3. If both rule 2 and rule 1 are applying, rule 2 takes precedence.
4. An instruction crossing a page boundary with differing `MTAG` value, the instruction behaves as if it was fetched from a page with MTAG=0.

`MTAG` bit in page table entry remains a reserved bit if `XWR == 111` or `XWR == 010` and if set, will raise a page fault of original access type.

If memory tagging is not enabled for the execution environment via *envcfg CSR, then `MTAG` bit in page table entry remains a reserved and if set will raise a page fault of original access type.

> *`MTAG` bit on data pages allows software to opt into selected memory regions to generate checked loads and stores. Furthermore, `MTAG` bit on executable pages allows software to opt out certain code regions from being subject to checked loads and stores. From usability point of view, shadow stack memory accesses or self modifying code do not need `MTAG` bit. Thus `MTAG` bit is kept as reserved for such page table encodings.*

### 2.7.1. Tag checks on supervisor accesses to user pages

Supervisor accesses to user pages do not generate checked accesses and if memory tagging is enabled for user mode then MTAG bit in first stage page tables is ignored.

## 2.8. Memory tagging and hypervisor memory accesses

HLVX* instructions always generate unchecked loads. HLV* and HSV* instructions `MTE_MODE` is defined based on privilege level at which they are executing and may generate checked memory accesses.

- HS mode: MTE_MODE for HLV*/HSV* is taken from `henvcfg.MTE_MODE`.
- HU mode: MTE_MODE for HLV*/HSV* is taken from `hstatus.HUMT`.

If page based virtual memory is enabled for V=1, HLV*/HSV* generate checked memory accesses based on rules specified by paging bit `MTAG` in first stage page tables (see Section 2.7).

If HLV*/HSV* instructions result in a tag mismatch, software check exception is delivered to HS mode with tval = 4 and hstatus.GVA set.

## 2.9. CSR bits for memory tagging

In M-mode, enable for memory tagging is controlled via `mseccfg` CSR.

Enablement for privilege modes less than M-mode is controlled through `xenvcfg` CSR. Zimt adds two bits termed as `MT_MODE` to `xenvcfg` CSR which controls enabling of memory tagging and `pointer_tag_width` for the next privilege mode. A `MT_ASYNC` bit (bit 36) is added to `xenvcfg` CSR and

if set, software check exceptions due to tag mismatches on store operations can be reported asynchronously (see Section 2.6).

### 2.9.1. Memory tagging enable and pointer_tag_width

The term `xMT_MODE` is used to determine if memory tagging is enabled in current execution environment (privilege mode).

Following table describes different encodings of `MT_MODE` and corresponding configuration

*Table 2.* `MT_MODE` *encoding and its meaning*

| `MT_MODE` | Memory tagging state |
|---|---|
| 00 | Disabled |
| 01 | Reserved |
| 10 | Enabled, pointer_tag_width = 4 |
| 11 | Enabled, pointer_tag_width = 7 |

```
If memory tagging is implemented, implementation must implement
`pointer_tag_width = 4` at minimum. To discover maximum supported
`pointer_tag_width`, software can write `0b11` to `MT_MODE` field in the
`__x__envcfg` CSR and read it back. If read back value is `0b11` then
implementation supports both `pointer_tag` widths.
```

```
If xMT_MODE == 0b00 then xenvcfg.MT_ASYNC becomes WPRI
```

### 2.9.2. Machine Security Configuration Register(`mseccfg`)

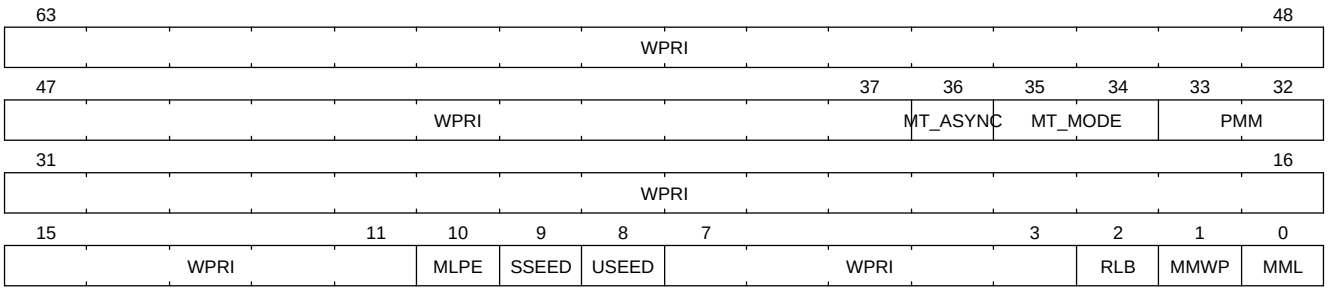


*Figure 1. Machine security configuration register(*`mseccfg`*)*

The Zimt extension adds the `MT_MODE` field (bit 35:34) to `mseccfg`. When the `MT_MODE` field is set to `0b10` or `0b11`, memory tagging is enabled for M-mode.

When `MT_MODE` is `0b00`, the following rules apply to M-mode:

- Zimt instructions will revert to their behavior as defined by Zimop.

### 2.9.3. Machine Environment Configuration Register (`menvcfg`)

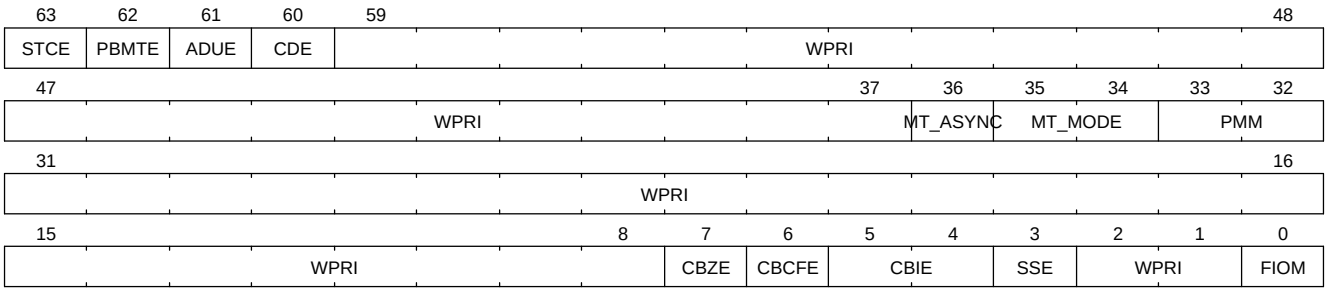


*Figure 2. Machine environment configuration register (`menvcfg`)*

The Zimt extension adds the `MT_MODE` field (bit 35:34) to `menvcfg`. When the `MT_MODE` field is set to `0b10` or `0b11`, memory tagging is enabled for HS/S-mode.

When `MT_MODE` is `0b00`, the following rules apply to HS/S-mode:

- Zimt instructions will revert to their behavior as defined by Zimop.

## 2.9.4. Supervisor Environment Configuration Register (`senvcfg`)

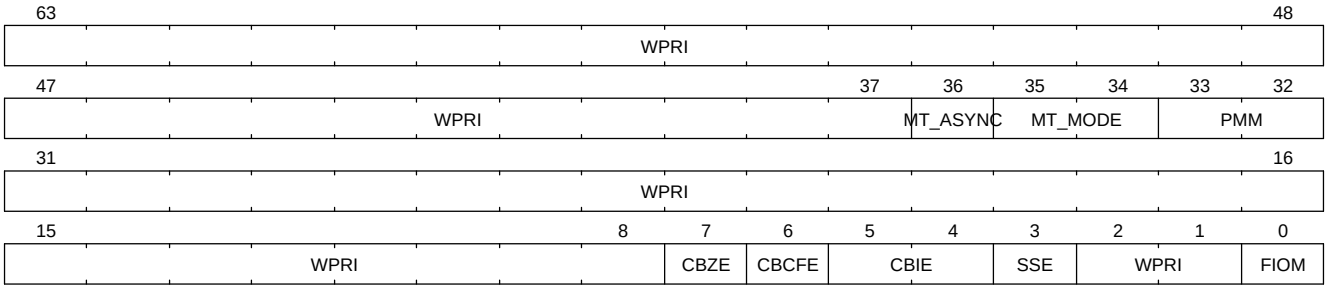


*Figure 3. Supervisor environment configuration register (`senvcfg`)*

The Zimt extension adds the `MT_MODE` field (bit 35:34) to `senvcfg`. When the `MT_MODE` field is set to `0b10` or `0b11`, memory tagging is enabled for VU/U-mode.

When `MT_MODE` is `0b00`, the following rules apply to VU/U-mode:

- Zimt instructions will revert to their behavior as defined by Zimop.

## 2.9.5. Hypervisor Environment Configuration Register (`henvcfg`)

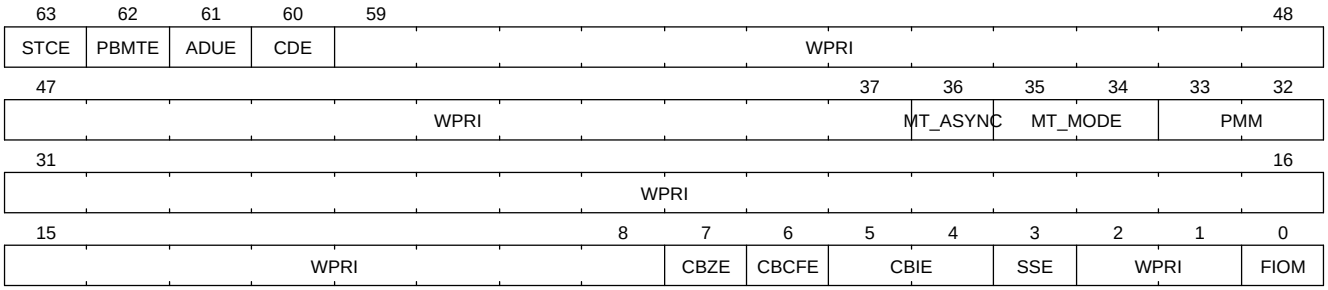


*Figure 4. Hypervisor environment configuration register (`henvcfg`)*

The Zimt extension adds the `MT_MODE` field (bit 35:34) to `henvcfg`. When the `MT_MODE` field is set to `0b10` or `0b11`, memory tagging is enabled for VS-mode.

When `MT_MODE` is `0b00`, the following rules apply to VS-mode:

- Zimt instructions will revert to their behavior as defined by Zimop.

### 2.9.6. Hypervisor Status Register (`hstatus`)

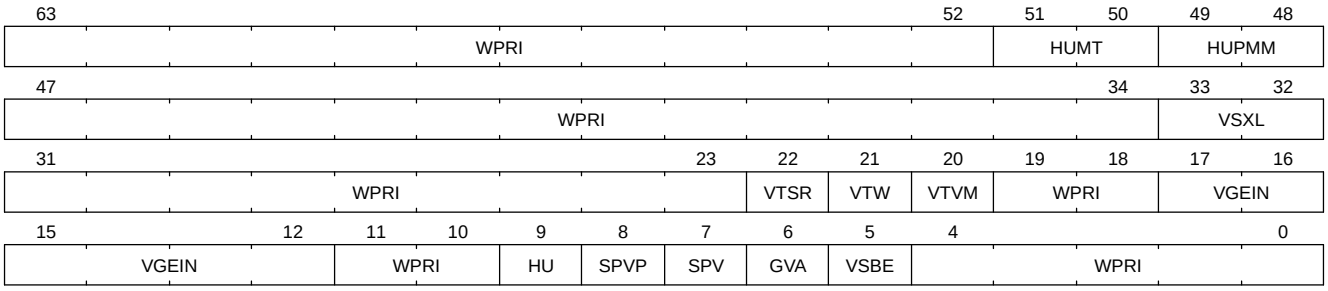


*Figure 5. Hypervisor status register (`hstatus`)*

The Zimte extension adds the `HUMT` field (bit 51:50) to `hstatus`. When the `HUMT` field is set to `0b10` or `0b11`, HLV*/HSV* may generate checked accesses in U mode.

When `HUMT` is `0b00`, HLV*/HSV* instructions do not generate checked accesses in U mode even if first stage paging tables have MTAG=1 in page table entry.

# 2.10. Appendix

## 2.10.1. Example: stack tagging codegen

```
function:
    # N.B. sp remains untagged at all times
    addi sp, sp, -512 # stack frame size of 512 bytes
    gentag t0, zero   # generate a pointer_tag in high bits of t0
     :
    # first object is tagged <random tag> + 1
    addi s1, sp, 16
    addtag t1, t0, 1  # tag_imm4 = 1
    or s1, s1, t1
     :
    # second object is tagged <random tag> + 2
    addi s2, sp, 32
    addtag t1, t0, 2  # tag_imm4 = 2
    or s2, s2, t1
     :
    # scope of second object starts, tag
    settag s2, 1
    # [...] do things with s2 while in scope
    # scope of second object ends, tag back to zero
    addi s2, sp, 16
    settag s2, 1
```

# Chapter 3. Virtually indexed tag storage (Svatag and Smvatag)

Svatag (for S/HS/VS/U/VU) and Smvatag (for M) extensions defines that tag storage and tag lookups for memory chunks (`mc`) as defined by Zimt (see Chapter 2) extension is in virtual address space of execution environment. Tag storage can be viewed as a large array (hereafter referred to as virtually indexed tag table or VITT) of the tags (`mc_tag`) with each tag width = `mc_tag_width` (see Chapter 2 for `mc_tag` and `mc_tag_width`). Base virtual address of this large array is determined differently depending on privilege mode (see Section 3.2) and hereafter referred as `xVITT_BASE`.

## 3.1. Tag look up in virtually indexed tag table

Virtual address location (`mc_tag_va`) of `mc_tag` for a given memory chunk located at virtual address `va` is looked up in VITT using the following formula

```
mc_tag_va = VITT_BASE + (((va >> 4) * mc_tag_width) >> 3)
```

*Table 3.* `mc_tag_va` *for each* `mc_tag_width`

| mc_tag_width | mc_tag_va |
|---|---|
| 4 | VITT_BASE + (va >> 5) |
| 8 | VITT_BASE + (va >> 4) |

When a regular load/store is performed on a virtual address `va` with tag checks (see Section 2.5) enforced then hart performs tag look up by performing a load tag operation on `mc_tag_va` for the corresponding `va` and extracts `mc_tag` from the loaded value at the virtual address `mc_tag_va`.

Tag loads are part of tag look up and can raise load page fault on `mc_tag_va`, if it doesn't have read permissions in page tables.

*In case of* `mc_tag_width` *= 4 bit, each single byte in tag storage is packed with two* `mc_tag` *for two consecutive memory chunks. Thus extracting a* `mc_tag` *requires masking and shifting of bits after the value is loaded from* `mc_tag_va`.

When `settag` instruction is executed for memory chunks(s) starting at the virtual address `va` (see Section 2.4) and memory tagging is enabled for current execution environment, then hart performs tag store by performing a store on `mc_tag_va` for the corresponding `va`.

Tag stores are generated from `settag` and can raise store page fault on `mc_tag_va` address, if it doesn't have write permissions in page tables.

*In case of* `mc_tag_width` *= 4 bit, each single byte in tag storage is packed with two* `mc_tag` *for two consecutive memory chunks. Thus storing a* `mc_tag` *requires read, modify (masking and shifting of bits) and store back to* `mc_tag_va`.

## 3.2. Tag table base address CSRs

Tag storage base virtual address (`xVITT_BASE`) is defined using privileged CSRs. Tag table base address must be 4K aligned. Following table list the privileged CSRs.

*Table 4. Privileged tag base address CSRs*

| Privilege level | CSR |
|---|---|
| U | `svittu` |
| S / HS | `svitts` |
| VS | `vsvitts` |
| VU | `vsvittu` |
| M | `mvitt` |

`Svatag` defines the `svittu`, `svitts`, `vsvittu` and `vsvitts` CSRs while `Smvatag` defines `mvitt`.

## 3.3. Protection of tag storage

Hart must raise access faults on regular loads and stores to virtual addresses spanning VITT range. Hart must also raise access fault on `settag` instruction if the memory chunk specified in `rs1` overlaps with VITT range. VITT range is defined as below

```
tag_memory_region_start = mc_tag_va(LOWEST_VADDR_CURR_PRIV)
tag_memory_region_end = mc_tag_va(HIGHEST_VADDR_CURR_PRIV)
LOWEST_VADDR_CURR_PRIV: Lowest virtual address in current privilege.
HIGHEST_VADDR_CURR_PRIV: Highest virtual address in current privilege.
```

*Table 5. Lowest and highest virtual addresses*

| Privilege level | LOWEST_VADDR_CURR_PRIV | HIGHEST_VADDR_CURR_PRIV |
|---|---|---|
| U | 0 | 2 ^ (VADDR_BITS - 1) - 1 |
| S / HS / VS | 2 ^ (VADDR_BITS - 1) | 2 ^ (VADDR_BITS) - 1 |
| M | `Impl. defined` | `Impl. defined` |

VADDR_BITS is virtual addressing bits. VADDR_BITS can be 39, 48 or 57 depending on Sv39, Sv48 or Sv57 virtual addressing mode in `satp` CSR. In bare mode and M-mode, LOWEST_VADDR_CURR_PRIV and HIGHEST_VADDR_CURR_PRIV are implementation defined.

`Svatag` depends on `Svukte` extension and thus assumes an even split in address space between user and supervisor with user address space spanning from zero to maxium positive address while supervisor address space spanning from minimum negative value to maximum negative value.

HLV* and HSV* instructions use `vsvittu` or `vsvitts` as base of the tag table depending on if `hstatus.SPVP=0` or `hstatus.SPVP=1`, respectively.

# Chapter 4. tag check elide extension

`Zitagelide` is defined to aid software to elide tag checks on memory accesses on a tagged page, and is dependent on `Zimt`, `C` and `Zcmop` extensions.

Certain regular load or store can be elided for tag checks if software (compiler) can statically determine that they are safe to access. One such situation is function locals where compiler can statically determine that memory access is not out of bounds or out of scope. Although pointers to function locals passed to another function will require tag checks. Thus page tables will mark such pages as tagged data page. To help software elide tag checks on tagged pages, `Zitagelide` introduces a new hart state `transient tag check disable` (TTCD) state and is defined as below:

- 0 - `TAG_CHECK_ENFORCED` - If memory tagging is enabled, tag checks enforced.
- 1 - `TAG_CHECK_ELIDE` - If memory tagging is enabled, tag checks elided.

`Zitagelide` defines a new instruction `nietc` short for next instruction elide tag check(s). Encoding for `nietc` is taken from `C.MOP.3`, thus making `Zimt` dependent on `C` and `Zcmop` extensions. If memory tagging is enabled, then instruction `nietc` sets `TTCD` to `TAG_CHECK_ELIDE` state. A subsequent regular memory load or store clears `TTCD` to `TAG_CHECK_ENFORCED`. Thus any regular load or store originating from subsequent instruction can be elided for tag checks and re-arms the hart to check for tags thereafter.

*Compiler can insert `nietc` before loads and stores which are accessing objects local to a function or container objects and gain performance back. It will lead to a code size growth (2 additional byte per load/store) but stack tagging will anyways lead to code size growth and it is expected that user enabling stack tagging has opted into code size growth as a trade-off.*

## 4.1. CSR bits for elide tag check

`Zitagelide` doesn't require any enabling bit. If memory tagging is enabled in the execution environment and an implementation implements `Zitagelide`, then software can leverage tag `nietc` instruction. If a trap occurs between `nietc` and subsequent instruction then `TTCD` state is spilled into *status register of the mode where trap is being delivered. Bit position of `TTCD` spilled state in *status register is defined as below.

### 4.1.1. Machine Status Register (`mstatus`)

| 63 | 62 … 48 | 47 … 43 | 42 | 41 | 40 | 39 | 38 | 37 | 36 | 35 … 34 | 33 … 32 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| SD | WPRI | | | | | | | | | | |
| | | WPRI | MPTTCD | MPELP | WPRI | MPV | GVA | MBE | SBE | SXL[1:0] | UXL[1:0] |

| 31 … 25 | 24 | 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|---|---|
| WPRI | SPTTCD | SPELP | TSR | TW | TVM | MXR | SUM | MPRV | XS[1:0] |

| 15 | 14 … 13 | 12 … 11 | 10 … 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| XS[1:0] | FS[1:0] | MPP[1:0] | VS[1:0] | SPP | MPIE | UBE | SPIE | WPRI | MIE | WPRI | SIE | WPRI |

*Figure 6. Machine-mode status register (`mstatus`) for RV64*

The Zimt extension introduces the `SPTTCD` (bit 24) and `MPTTCD` (bit 42) fields that hold the previous `TTCD`. The `xPTTCD` fields are encoded as follows:

- 0 - `TAG_CHECK_ENFORCED` - If memory tagging is enabled, tag checks enforced.
- 1 - `TAG_CHECK_ELIDE` - If memory tagging is enabled, tag checks elided.

On a mret/sret, MPTTCD/SPTTCD is restored into hart's `TTCD` and MPTTCD/SPTTCD is cleared.

### 4.1.2. Supervisor Status Register (`sstatus`)

| 63 | 62 … 48 | 47 … 34 | 33 … 32 | 31 … 25 | 24 | 23 | 22 … 20 | 19 | 18 | 17 | 16 | 15 | 14 … 13 | 12 … 11 | 10 … 9 | 8 | 7 | 6 | 5 | 4 … 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SD | WPRI | WPRI | UXL[1:0] | WPRI | SPTTCD | SPELP | WPRI | MXR | SUM | WPRI | XS[1:0] | XS[1:0] | FS[1:0] | WPRI | VS[1:0] | SPP | WPRI | UBE | SPIE | WPRI | SIE | WPRI |

*Figure 7. Supervisor-mode status register (`sstatus`) when `SXLEN=64`*

Access to the `SPTTCD` field introduced by Zimt accesses the homonymous fields of `mstatus` when `V=0` and the homonymous fields of `vsstatus` when `V=1`.

On a sret, SPTTCD is restored into hart's `TTCD` and SPTTCD is cleared.

### 4.1.3. Virtual Supervisor Status Register (`vsstatus`)

| 63 | 62 … 48 | 47 … 34 | 33 … 32 | 31 … 25 | 24 | 23 | 22 … 20 | 19 | 18 | 17 | 16 | 15 | 14 … 13 | 12 … 11 | 10 … 9 | 8 | 7 | 6 | 5 | 4 … 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SD | WPRI | WPRI | UXL[1:0] | WPRI | SPTTCD | SPELP | WPRI | MXR | SUM | WPRI | XS[1:0] | XS[1:0] | FS[1:0] | WPRI | VS[1:0] | SPP | WPRI | UBE | SPIE | WPRI | SIE | WPRI |

*Figure 8. Virtual supervisor status register (`vsstatus`) when `VSXLEN=64`*

The Zimt extension introduces the `SPTTCD` (bit 24) field that hold the previous `TTCD`. The SPTTCD fields is encoded as follows:

- 0 - `TAG_CHECK_ENFORCED` - If memory tagging is enabled, tag checks enforced.
- 1 - `TAG_CHECK_ELIDE` - If memory tagging is enabled, tag checks elided.

# Bibliography

[1] "LLVM StackColoring.cpp." [Online]. Available: github.com/llvm/llvm-project/blob/main/llvm/lib/CodeGen/StackColoring.cpp.

[2] "cppreference.com - Scope." [Online]. Available: en.cppreference.com/w/cpp/language/scope.html.

[3] "HWAddress Sanitizer." [Online]. Available: clang.llvm.org/docs/HardwareAssistedAddressSanitizerDesign.html.

[4] "Address Sanitizer." [Online]. Available: github.com/google/sanitizers/wiki/AddressSanitizer.

[5] "RISC-V Pointer Masking." [Online]. Available: github.com/riscv/riscv-j-extension/blob/master/zjpm-spec.pdf.

[6] "Zimop May-Be-Operations Extension." [Online]. Available: github.com/riscv/riscv-isa-manual/blob/main/src/zimop.adoc.